LÓGICA COMPUTACIONAL



SOLUÇÕES EDUCACIONAIS INTEGRADAS

Álgebra booleana

Marcelo da Silva dos Santos

OBJETIVOS DE APRENDIZAGEM

- Definir expressões booleanas.
- Analisar representações com mintermos e maxtermos.
- Reconhecer a importância do mapa de Karnaugh.

Introdução

A álgebra booleana é caracterizada por um conjunto análogo de ferramentas, metodologias e processos matemáticos oriundos da álgebra clássica, mas aplicados a variáveis lógicas, ou seja, aquelas cuja avaliação resulta em um valor lógico (V ou F, 1 ou 0, ligado ou desligado, dentre outras formas de definição). Essas proposições, reunidas em forma de expressões lógicas, permitem avaliar o relacionamento de duas ou mais condições em uma mesma condição (lógica simbólica, criação de circuitos, dentre outras aplicações para expressões).

Sua criação deve-se a George Boole, o qual também inspirou o conceito de variáveis booleanas, capazes de assumir somente valores lógicos. Este foi um dos seus trabalhos em lógica simbólica, o qual inspirou o trabalho de outros pesquisadores, como Edward W. Veitch e Maurice Karnaugh, responsáveis pela definição e pelo aprimoramento dos mapas de Karnaugh, uma importante ferramenta para otimização de expressões booleanas, bastante utilizados na eletrônica na construção de circuitos lógicos.

Para definição dessas expressões, são aplicadas regras bastante conhecidas na álgebra clássica, como as propriedades matemáticas (distributiva, comutativa, por exemplo), aplicadas na avaliação e simplificação de expressões. Ainda na definição de expressões podemos citar os métodos de definição por soma de produtos e produto de somas, bem como o já citado mapa de Karnaugh.

Neste capítulo, você será apresentado às expressões booleanas, base para a criação de circuitos ou realização de operações lógicas, por exemplo. Também verá como representar expressões através de mintermos e maxtermos, e como são os processos de simplificação de expressões, seja por meio de propriedades algébricas ou utilizando mapas de Karnaugh.

Conceitos de expressões booleanas

Álgebra booleana é um conjunto de operações e axiomas utilizados para representar a lógica através de equações algébricas. A metodologia foi introduzida por George Boole (TOCCI; WIDMER; MOSS, 2011), em meados do século XIX. Diferente da álgebra conhecida até então, Boole sugeriu que as variáveis poderiam assumir somente dois estados: o verdadeiro e o falso, também representado como 1 e 0, ligado ou desligado, dentre outras representações para o estado binário. Com este conjunto finito de valores, também podemos descrever uma expressão booleana através de tabelas-verdade, ou seja, uma representação da expressão em que são apresentadas as possíveis combinações de valores entre as variáveis de entrada e os valores de verdade que podem ser obtidos como saídas (DAGHLIAN, 1995).

Em 1938, Claude Shannon, até então um estudante no MIT (Massachusetts Institute of Technology de Boston, um renomado centro tecnológico situado nos Estados Unidos) correlacionou a álgebra booleana como circuitos eletrônicos, comparando os dois estados lógicos (verdadeiro e falso) com diferenças de potência elétrica em um circuito eletrônico (DAGHLIAN, 1995). Este foi um dos passos para a aplicação da álgebra de Boole no projeto de circuitos eletrônicos (como *microchips*) que, por meio de interações entre as funções lógicas, são capazes de descrever resultados em linguagem binária.

As operações realizadas por meio da álgebra booleana são baseadas em três operadores básicos: E, OU e NÃO (traduzido do inglês *AND*, *OR* e *NOT*). Com essas três funções básicas é possível efetuar comparações ou realizar as quatro operações aritméticas base. A seguir, você vai conhecer um pouco mais sobre cada uma delas.

Adição lógica (OU lógico)

Uma operação bem conhecida em formalização lógica é o OU lógico (*OR*), também conhecido como disjunção lógica. É uma operação binária, ou seja, ela relaciona duas variáveis lógicas, e sua saída será verdadeira sempre que

ao menos uma das variáveis possuir o valor verdadeiro, assim como para que a saída tenha o valor falso, todas as variáveis de entrada devem ser falsas.

O símbolo clássico de disjunção lógica é o V utilizado para indicar esta relação, por exemplo A V B (leia-se A ou B, A disjunção com B). Em álgebra booleana, essa operação é associada à operação de adição e seu símbolo mais comum é o "+", tal como na álgebra clássica, sendo então que A V B passa a ser representado como A + B.

Agora que você sabe como representar, podemos passar para a operação algébrica em si. Como sabemos, estamos trabalhando com adições lógicas e não com uma adição clássica. Assim, representamos os resultados das expressões como são realizados em uma tabela-verdade. Observe no exemplo da Figura 1.

A	B	$A \vee B$
1	1	1
1	0	1
0	1	1
0	0	0

$\boldsymbol{A + B}$
1 + 1 = 1
1 + 0 = 1
0 + 1 = 1
0 + 0 = 0

Figura 1. Comparativo entre uma operação lógica OU (a) e uma adição algébrica booleana (b).
Fonte: Adaptada de Abe (2002) e Tocci, Widmer e Moss (2011).

Produto lógico (E lógico)

A formalização lógica é representada pelo E lógico (*AND*), também conhecido como conjunção lógica. Também é representada por uma operação binária, ou seja, para que a saída resultante da operação entre as duas variáveis lógicas seja verdadeira, todos os valores de entrada devem obrigatoriamente possuir o valor verdadeiro, assim como para que a saída tenha o valor falso, ao menos uma das variáveis de entrada deve ser falsa.

O símbolo clássico de conjunção lógica é o ^ utilizado para indicar essa relação; por exemplo $A \wedge B$ (leia-se A e B, A conjunção com B). Em álgebra booleana, essa operação é associada à operação de multiplicação e seu símbolo mais comum é o ".", tal como na álgebra clássica, sendo então que $A \wedge B$ passa a ser representado como $A \cdot B$.

Assim como na adição, também representa a multiplicação de forma lógica, agora associada à respectiva tabela-verdade para a conjunção lógica.

Observe na Figura 2 o relacionamento entre a tabela-verdade e a operação de multiplicação lógica.

A	B	$A \wedge B$
1	1	1
1	0	0
0	1	0
0	0	0

(a)

$\boldsymbol{A \,.\, B}$
1 . 1 = 1
1 . 0 = 0
0 . 1 = 0
0 . 0 = 0

(b)

Figura 2. Comparativo entre uma operação lógica E (a) e uma multiplicação (produto) algébrica booleana (b).
Fonte: Adaptada de Abe (2002) e Tocci, Widmer e Moss (2011).

Complementação ou inversão lógica (NÃO lógico)

Ao contrário das anteriores, esta operação pode ser aplicada individualmente às variáveis, ou seja, ela é uma operação unária. Em formalização lógica, é representada pelo NÃO lógico (*NOT*) e possui a função de complementar (ou inverter) o valor lógico de uma entrada, ou seja, caso a entrada seja verdadeira, seu valor complementar (a saída invertida) será falso e vice-versa.

Seu símbolo clássico é ¬ ou o ~ colocado antes da variável, por exemplo ¬A ou ~B (leia-se não A e não B, respectivamente). Em álgebra booleana, sua representação mais comum é realizada por meio de um traço sobre a variável ou expressão que deve ter sua saída invertida, desta forma A. Observe na Figura 3 o relacionamento entre a tabela-verdade e a operação de complementação lógica.

A	$\neg A$
0	1
1	0

(a)

$\boldsymbol{\overline{A}}$
0 = 1
1 = 0

(b)

Figura 3. Comparativo entre uma negação lógica *NOT* (a) e uma complementação algébrica booleana (b).
Fonte: Adaptada de ABE (2002) e Tocci, Widmer e Moss (2011).

Cálculo de uma expressão booleana e relação entre operadores

Ao avaliar uma expressão booleana, vamos notar mais semelhanças com a álgebra clássica, como as regras de precedência de operações (DAGHLIAN, 1995). Tal como em expressões em álgebra clássica, quando encontramos na mesma expressão operações de multiplicação e adição, devemos seguir uma ordem específica para a resolução. Considerando que avaliamos a expressão da esquerda para à direita:

- primeiro devem ser executadas as operações entre parênteses;
- em segundo lugar, devem ser executadas as operações de multiplicação (produto);
- por último, devem ser executadas as operações de adição.

Quando a expressão possuir funções de complementação, ela deve ser aplicada tão logo seja possível. Já no caso de ser aplicada a complementação (negação) sobre uma subexpressão inteira (A+B), primeiro obtenha o resultado para a subexpressão e, logo após, inverta o resultado.



Fique atento

Expressões booleanas podem ser representadas por meio de tabelas-verdade, mas nem sempre sua construção será trivial. Considerando que o tamanho de uma tabela segue um crescimento exponencial condicionado pela fórmula 2^n (onde 2 representa o caráter binário e *n* o número de variáveis lógicas), o número de linhas necessário pode se tornar inviável sem a utilização de um computador.

Considere que uma expressão com duas variáveis possuirá $2^2 = 4$ linhas de altura, mas para cinco variáveis teremos $2^5 = 32$ linhas e para dez variáveis (2^{10}) chegaremos a 1024 linhas, tornando extremamente complexo o controle de todas as combinações possíveis.

Para facilitar a compreensão, considere a expressão $X + Y . \bar{Z}$, onde podemos observar três variáveis de entrada (X, Y e Z). Sabemos que o número de combinações que pode ser obtido para esta expressão é 8 (2^3), logo a tabela-verdade possui oito linhas. Quanto às colunas, são inseridas à direita uma coluna para cada variável inicial e ao menos uma coluna ao final, para registrar o resultado da expressão. Podemos utilizar colunas intermediárias para os cálculos parciais e, à medida que você for aprendendo, poderá utilizar uma menor quantidade de cálculos parciais. No exemplo (Quadro 1) utilizaremos mais colunas para que você perceba todas as operações realizadas.

Quadro 1. Exemplo de tabela-verdade construída para a expressão booleana $X + Y . \bar{Z}$

X	Y	Z	$\bar{Z}$	$Y.\bar{Z}$	$X + Y \cdot \bar{Z}$
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	0	0	0
1	0	0	1	0	1
1	0	1	0	0	1
1	1	0	1	1	1
1	1	1	0	0	1

Fonte: Adaptado de Tocci, Widmer e Moss (2011).

Nesta expressão, como não há parênteses, vamos iniciar pelas subexpressões de multiplicação, no caso $Y . \bar{Z}$ (uma operação E). Também vamos incluir uma coluna para acomodar o resultado da complementação de Z. Finalmente, vamos realizar a adição (operação OU) do resultado da subexpressão com a variável X ($X + (Y . \bar{Z})$. Na última coluna, podemos observar os possíveis resultados para expressão booleana, em relação a cada possibilidade de entrada.

Representações com mintermos e maxtermos

Anteriormente, você acompanhou como construir uma tabela-verdade a partir de uma expressão booleana, mas também é possível realizar o processo contrário, ou seja, a partir de uma tabela-verdade qualquer, identificar a expressão booleana capaz de descrevê-la. Podemos utilizar duas estratégias para chegar a essa expressão: o método da soma dos produtos ou o método do produto das somas.

Nesta seção, vamos descrever melhor esses processos e como aplicá-los para obter as expressões algébricas correspondentes.

Soma de produtos (mintermos)

O primeiro método estudado será a soma de produtos, que, como o próprio nome já indica, realiza um somatório (adições obtidas através de funções OU) de todos os produtos (multiplicações obtidas através de uma função E) de uma tabela-verdade. Para tanto, a tabela deve ser completa, ou seja, todas as variáveis envolvidas, complementadas (negadas) ou não, devem participar da avaliação.

Em nosso exemplo, vamos utilizar uma tabela composta por três variáveis lógicas representadas pelas letras A, B e C, que representam todos os valores de entrada. Para cada conjunto de entradas, ou seja, cada linha da tabela, é indicado um valor de saída, que pode ser 0 ou 1. Na indústria, por exemplo, cada saída pode representar uma ação que um circuito eletrônico deve executar, e o valor (0 ou 1) que cada saída assume para cada conjunto de entradas vai depender do objetivo do circuito. Como estamos apenas exemplificando o método de soma de produtos, os valores de saída em nossa tabela de exemplo são aleatórios e não representam nenhuma ação específica.

Como próximo passo, vamos identificar o produto de cada linha da tabela. Este processo é simples: para cada linha, vamos substituir os valores de entrada pelas letras que representam as variáveis, sendo que para cada valor 1, representaremos pela própria letra, e a cada valor 0, representaremos pela negação da variável. Para concluir, vamos unir as variáveis através de multiplicações. Assim, vamos ter um termo produto, mais conhecido como **mintermo**. Observe o exemplo na Figura 4.

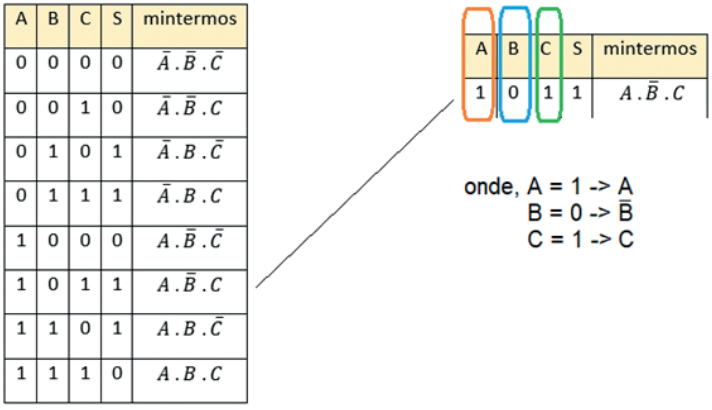


Figura 4. Tabela-verdade apresentando as três variáveis de entrada (A, B e C), os valores de saída (coluna S) e os mintermos respectivos para cada conjunto de entradas. Ao lado, é apresentado o método de tradução para cada termo, sendo realizada a transcrição de cada variável, dependendo do valor da entrada.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

O próximo passo será a identificação de quais termos devem compor a expressão. Para isso, vamos selecionar somente os mintermos onde a saída seja igual a 1 e, a partir deste conjunto, realizar um somatório desses mintermos (Figura 5).

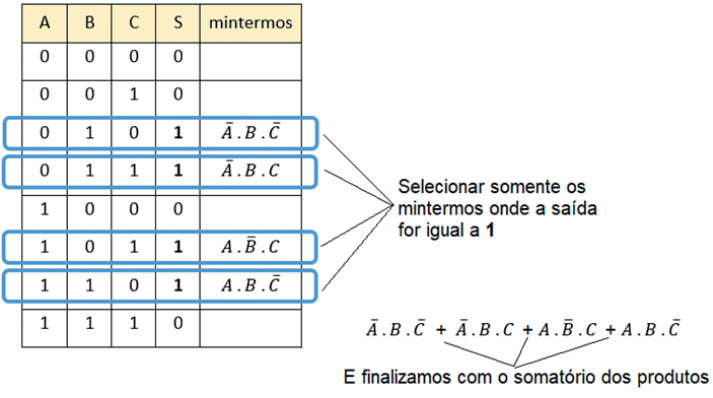


Figura 5. Após a identificação dos mintermos que irão compor a expressão booleana que representa a tabela (linhas onde as saídas são iguais a 1), finalizamos com o somatório dos termos.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

Com base na construção até então realizada, chegamos a quatro mintermos, representados pelos produtos $\bar{A} . B . \bar{C}$, $\bar{A} . B . C$, $A . \bar{B} . C$ e $. B . \bar{C}$. Para finalizar, basta realizar o somatório dos produtos, obtendo então a expressão booleana $\bar{A} . B . \bar{C} + \bar{A} . B . C + A . \bar{B} . C + A . B . \bar{C}$, que representa a tabela-verdade apresentada. Ainda podemos simplificar a fórmula, considerando que a álgebra nos permite ocultar o operador de multiplicação, resultado na expressão equivalente $\bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C + AB\bar{C}$.

Produto das somas (maxtermos)

O método de produto das somas tem características bem semelhantes àquelas que você acompanhou até agora, mas aplicando as operações ao contrário. Agora vamos aplicar um produtório (multiplicações obtidas através de funções E) de todas as somas (adições obtidas através de funções OU). Aqui a tabela também deve ser completa, ou seja, todas as variáveis envolvidas, complementadas (negadas) ou não, devem participar da avaliação. Vamos utilizar como exemplo a mesma tabela-verdade do exemplo anterior, ou seja, saiba que podemos extrair de uma mesma tabela duas expressões não idênticas, mas equivalentes, capazes de representar a tabela-verdade.

Aqui começam as diferenças: cada termo será criado através da soma das variáveis. Ao contrário do método anterior, agora a tradução é realizada substituindo as entradas com valor 1 pela variável negada e as entradas com valor 0 pela variável em si. Além disso, agora vamos unir os termos utilizando adições. Os termos criados desta forma são conhecidos como **maxtermo**. Observe o processo de construção dos maxtermos na Figura 6.

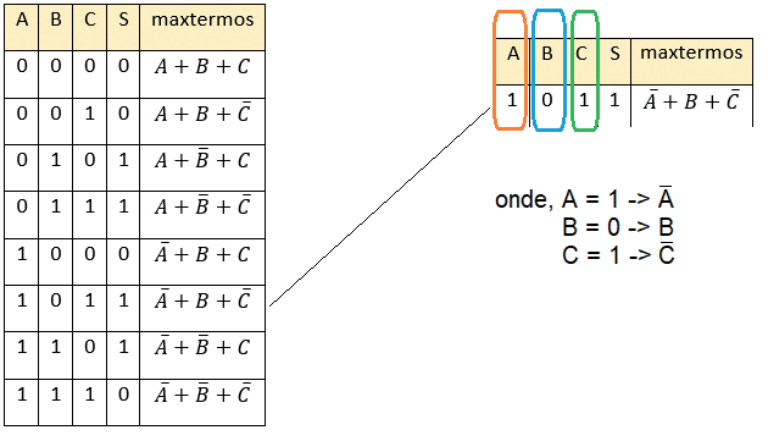


Figura 6. Tabela-verdade apresentando as três variáveis de entrada (A, B e C), os valores de saída (coluna S) e os maxtermos respectivos para cada conjunto de entradas. Ao lado, é apresentado o método de tradução para cada termo, sendo realizada a transcrição de cada variável, dependendo do valor da entrada.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

O próximo passo será a identificação de quais termos devem compor a expressão. Como agora estamos utilizando maxtermos, ao contrário do método anterior, vamos buscar todas as saídas iguais a 0 (Figura 7).

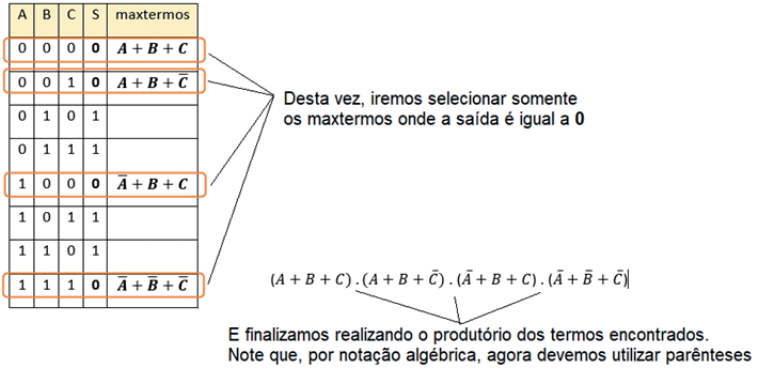


Figura 7. Após a identificação dos maxtermos que vão compor a expressão booleana que representa a tabela (linhas onde as saídas são iguais a 0), finalizamos com o produtório (multiplicação) dos termos.
Fonte: Adaptada Tocci, Widmer e Moss (2011).

Com base na construção até então realizada, chegamos também a quatro maxtermos, representados pelos somatórios $A + B + C, A + B + \bar{C}, \bar{A} + B + C$ e $\bar{A} + \bar{B} + \bar{C}$. Para finalizar, basta realizar a multiplicação desses termos, obtendo então a expressão booleana $(A + B + C) . (A + B + \bar{C}) . (\bar{A} + B + C) . (\bar{A} + \bar{B} + \bar{C})$, que representa a tabela-verdade apresentada. Também podemos simplificar a expressão ocultando o operador de multiplicação, resultando na expressão equivalente $(A + B + C)(A + B + \bar{C})(\bar{A} + B + C)(\bar{A} + \bar{B} + \bar{C})$.

Propriedades da álgebra booleana e simplificação de expressões

Como já foi comentado na seção anterior, assim como na álgebra clássica, a álgebra booleana também segue algumas propriedades clássicas deduzidas a partir das características das operações algébricas. As expressões vistas até então estão em sua forma completa (também chamadas de expressões canônicas), ou seja, cada mintermo ou maxtermo apresenta todas as variáveis utilizadas em sua construção (TOCCI; WIDMER; MOSS, 2011). Isto implica termos redundantes e, consequentemente, a remoção dessas redundâncias vai gerar economia de elementos na implementação das expressões em circuitos eletrônicos.

Antes de aprender a simplificar, vamos conhecer algumas definições elementares. Vamos numerar as regras meramente para auxiliar em sua identificação quando precisarmos referenciar algumas delas em específico (DAGHLIAN, 1995). Primeiro, observe as regras algébricas básicas referentes às operações básicas. São elas:

- Adição lógica:
  - (1) $A + 0 = A$
  - (2) $A + 1 = 1$
  - (3) $A + A = A$
  - (4) $A + \bar{A} = 1$
- Multiplicação lógica:
  - (5) $A . 0 = 0$
  - (6) $A . 1 = A$
  - (7) $A . A = A$
  - (8) $A . \bar{A} = 0$

- Complementação ou negação:
  - (9) $\bar{\bar{A}} = A$ (o traço duplo sobre a variável lógica é equivalente a uma dupla negação)



Saiba mais

De Morgan também estendeu suas leis de complementação para expressões booleanas — temos, em seu primeiro teorema, que a complementação de um produto equivale à soma das negações de cada variável do produto, gerando a expressão:

$$\overline{A.B.C.\cdots n} = \bar{A} + \bar{B} + \bar{C} + \cdots \bar{n}$$

Já no segundo teorema temos o contrário, ou seja, que a complementação da soma equivale ao produto das negações individuais de cada variável:

$$\overline{A + B + C + \cdots n} = \bar{A}.\bar{B}.\bar{C}.\cdots \bar{n}$$

Assim como na álgebra clássica, ao operar com mais termos (ou variáveis), podemos decompor a equação em pares, ou seja, para uma expressão *A* + *B* + *C*, podemos primeiro calcular o resultado de *A* + *B* e depois calcular o valor resultante com os valores de *C*. Esta propriedade é chamada de associativa. Outra característica é que, já que estamos trabalhando somente com adições, é irrelevante a ordem das operações, ou seja, (*A* + *B*) + *C* ou *A* + (*B* + *C*), sendo esta propriedade chamada de comutativa.

- Comutatividade:
  - (10) $A + B = B + A$
  - (11) $A.B = B.A$
- Associatividade:
  - (12) $A + (B + C) = (A + B) + C = (A + C) + B$
  - (13) $A.(B.C) = (A.B).C = (A.C).B$
- Distributiva (da multiplicação em relação à adição):
  - (14) $A.(B + C) = A.B + A.C$

Como exemplo de simplificação utilizando as propriedades algébricas, considere a expressão canônica $S = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}C$.

- Uma das regras bastante úteis em simplificação é a propriedade distributiva, na qual podemos colocar um termo em evidência e reduzir a quantidade de ocorrências na expressão. Para tanto, vamos buscar por redundâncias nas variáveis. Note que o primeiro termo ($\bar{A}\bar{B}\bar{C}$) e o segundo termo ($\bar{A}B\bar{C}$) possuem as variáveis $\bar{A}\bar{C}$. Assim, pela regra 14, chegamos à expressão $S = \bar{A}\bar{C}.(B + \bar{B}) + A\bar{B}C$..
- Na expressão resultante, podemos agora tomar a expressão $B + \bar{B}$, que pela regra 4 sabemos que podemos substituir por 1, obtendo $\bar{A}\bar{C}$.
- Pela regra 6 da multiplicação (produto lógico), podemos substituir $\bar{A}\bar{C}.(1)$ por $\bar{A}\bar{C}$, chegando à expressão simplificada $\boldsymbol{S = \overline{AC} + A\overline{B}C}$.

Com isso, vamos economizar componentes na construção de circuitos e simplificar sua atuação. Observe outro exemplo, agora com uma expressão não canônica, ou seja, não completa, representada por $S = ABC + A\bar{C} + A\bar{B}$:

- Novamente, vamos utilizar a propriedade distributiva, pois encontramos a variável *A* nos três termos. Assim, colocando a variável em evidência, chegamos a $A(BC + \bar{C} + \bar{B})$.
- No interior dos parênteses, possuímos uma multiplicação BC e os mesmos termos em forma negada. Para simplificar a organização, utilizamos a propriedade associativa, chegando assim a $A(BC + (\bar{C} + \bar{B}))$.
- Parece que chegamos ao final, mas ainda podemos aproveitar algumas características para ir além. Por exemplo, para que pudéssemos eliminar as variáveis B e C, necessitaríamos que a operação entre parênteses $(\bar{C} + \bar{B})$ fosse uma multiplicação. Utilizando o teorema de Morgan, onde $\bar{C} + \bar{B}$ é equivalente a $\overline{CB}$, podemos definir que $A(BC + \overline{CB})$. Observação: pela regra da comutatividade (regra 11), $\overline{CB}$ e $\overline{BC}$ são expressões equivalentes.
- Agora podemos aplicar a regra 4 ($A(1)$) e a regra 6 ($A.1 = A$), chegando à expressão final $\boldsymbol{S = A}$.

Neste último exemplo, notamos que as variáveis B e C não tinham impacto algum na expressão, importando somente o valor de A, reforçando a relevância de obter expressões mais simples. Apesar de importante, também notamos que a simplificação através das propriedades algébricas pode se tornar morosa e complexa, dependendo do volume de variáveis envolvidas. Para tanto, existem métodos mais apropriados para simplificar expressões canônicas, como o método do mapa de Karnaugh (DAGHLIAN, 1995).

Mapas de Karnaugh

As expressões em suas formas completas, ou seja, onde cada termo possui todas as variáveis, é chamada de forma padrão ou expressão canônica. Entretanto, esse formato de apresentação pode apresentar redundâncias em alguns termos ou mesmo termos que não têm impacto para o resultado do circuito lógico. Essas redundâncias podem, além de aumentar o custo em caso de construção de um circuito eletrônico baseado na expressão, reduzir sua eficiência ou o seu tempo de resposta. Para resolver essa situação, conhecemos o processo de simplificação de expressões utilizando as propriedades da álgebra aplicadas sobre expressões lógicas. Mas este processo, além de complexo, é passível de falhas, visto que podemos parar de simplificar antes de chegar à expressão mais sucinta (TOCCI; WIDMER; MOSS, 2011).

O mapa de Karnaugh é um dos métodos mais eficientes para simplificação de expressões, mais simples e exato do que aplicar diretamente as propriedades algébricas das expressões booleanas. Ele foi descrito pela primeira vez por Edward Veitch, na metade do século XIX, e, posteriormente, o engenheiro de telecomunicações Maurice Karnaugh o aperfeiçoou para o formato conhecido até hoje, recebendo o seu nome.

O método é baseado na reconstrução da tabela-verdade por meio da identificação visual de agrupamentos de mintermos candidatos à simplificação (DIAS, 2013). Apesar de ser menos abordado na literatura, o método também pode ser aplicado utilizando maxtermos. Como parte do processo de construção, necessitamos reorganizar a disposição dos termos de forma a facilitar o processo. Como exemplo, vamos utilizar a mesma tabela utilizada na última seção.

A	B	C	S	mintermos	
0	0	0	0	$\bar{A}.\bar{B}.\bar{C}$	m_0
0	0	1	0	$\bar{A}.\bar{B}.C$	m_1
0	1	0	**1**	$\bar{A}.B.\bar{C}$	m_2
0	1	1	**1**	$\bar{A}.B.C$	m_3
1	0	0	0	$A.\bar{B}.\bar{C}$	m_4
1	0	1	**1**	$A.\bar{B}.C$	m_5
1	1	0	**1**	$A.B.\bar{C}$	m_6
1	1	1	0	$A.B.C$	m_7

Figura 8. Tabela-verdade com três variáveis e valores de saída S. Também podemos observar os mintermos formados a partir das entradas e os códigos de ordem que representam cada mintermo, representados pela letra m minúscula e pelo número sequencial iniciando em 0.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

Para construirmos o mapa, vamos configurar a tabela no formato de matriz, onde a coluna que apresenta as saídas será transposta como linhas dessa matriz (Figura 9). Considerando a construção de um mapa para uma expressão com *n* variáveis, será necessário construir uma tabela com 2^n células, no seguinte formato:

- para n = 2, gera-se uma matriz de 2 linhas e 2 colunas;
- para n = 3, gera-se uma matriz de 2 linhas e 4 colunas;
- para n = 4, gera-se uma matriz de 4 linhas e 4 colunas;
- para n = 5, gera-se uma matriz de 4 linhas e 8 colunas.

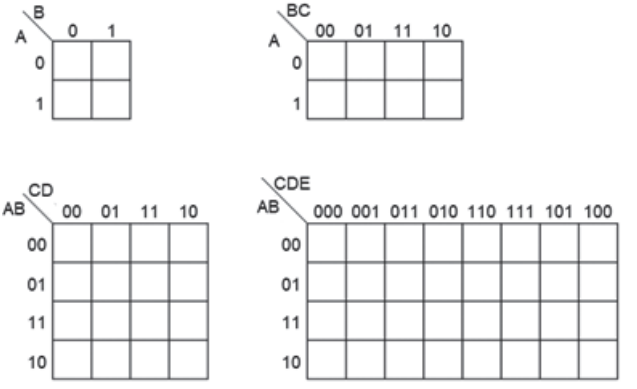


Figura 9. Exemplos de construção das grades para mapas de Karnaugh para 2, 3, 4 e 5 variáveis.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

Note que, para cada mapa, são rotuladas as variáveis para linhas ou colunas; a escolha de qual variável ou grupo de variáveis ocupa a linha ou coluna é irrelevante, o importante é posteriormente posicionar cada elemento no local correspondente. A divisão é realizada conforme a construção da tabela-verdade; por exemplo, para três variáveis, podemos dividi-las em dois grupos, sendo possíveis combinações A e BC (como na Figura 10) ou AB e C. Cada cabeçalho recebe uma quantidade correspondente de bits, ou seja, caso a linha ou coluna represente AB, ela receberá pares do tipo 00, 01, 11, 10; já a linha ou coluna que representa C, receberá um bit, sendo 0 e 1 seus valores.



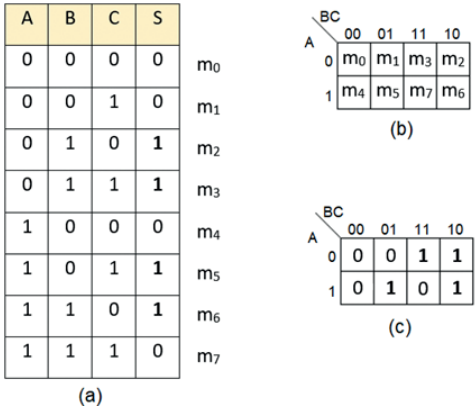
Exemplo

O código de Gray é um sistema baseado no código binário, descrito por Frank Gray (DORAN, 2007). Esta disposição teve seu surgimento ainda na época dos sistemas de válvulas termiônicas e dispositivos eletromecânicos. Tais componentes necessitavam de alta potência, além de gerar muito ruído nos momentos em que um conjunto próximo de bits se modificava simultaneamente. O código criado prevê que essas mudanças de bits devem ser de apenas um bit.

Código decimal	Código binário	Código Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100

Esta sequência é utilizada, por exemplo, na construção dos mapas de Karnaugh, mantendo ativa a busca por transições simplificadas e mais rápidas. Com base nessa sequência, existe a alteração da ordem das sequências binárias. Essa é, na verdade, a essência do mapa de Karnaugh. Esse processo seria o equivalente a você posicionar lado a lado os termos com mais variáveis em comum na álgebra booleana, simplificando, assim, o processo.

É importante ressaltar que a ordem desses bits deve seguir a ordem de Gray, logo será necessário invertermos alguns valores para que se adeque a este formato. Essa ordem é essencial para que a tabela tenha seu resultado correto.



A	B	C	S	
0	0	0	0	m_0
0	0	1	0	m_1
0	1	0	**1**	m_2
0	1	1	**1**	m_3
1	0	0	0	m_4
1	0	1	**1**	m_5
1	1	0	**1**	m_6
1	1	1	0	m_7

(a)

A \ BC	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

(b)

A \ BC	00	01	11	10
0	0	0	**1**	**1**
1	0	**1**	0	**1**

(c)

Figura 10. Processo de construção do mapa de Karnaugh: (a) tabela-verdade e indicação da ordem dos mintermos, representados por m minúsculo e número serial iniciando em 0; (b) esquema mostrando a posição que cada valor de saída deverá ocupar no mapa montado, note que como invertemos os binários 10 e 11, os valores transpostos devem seguir o mesmo padrão; (c) mapa finalizado, com valores posicionados.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

Após a construção da matriz, podemos começar a organizar as saídas na grade transpondo os valores da coluna de saída para cada posição respectiva do mapa. Na Figura 10, podemos observar que a sequência das linhas 3 e 4, bem como das linhas 7 e 8 estão invertidas. Isso se deve ao fato de buscarmos a menor variação de bits possível para a criação do mapa, sendo que o mesmo ocorre para as linhas 7 e 8.

Após a conclusão da etapa de construção do mapa, podemos então avaliar as expressões. Este é um processo simples, que inicia pela identificação de agrupamentos de mintermos. É importante observar algumas regras para formação desses grupos:

- O tamanho dos grupos deve seguir 2^n, ou seja, um mintermo isolado pode ser considerado um grupo, os demais seguem esse formato de crescimento (pares, quartetos, octetos, dentre outros).
- Os grupos devem reunir o maior número possível de mintermos, de forma a finalizar com a menor quantidade possível de grupos. Um elemento pode ser considerado em mais de um grupo (Figura 11).

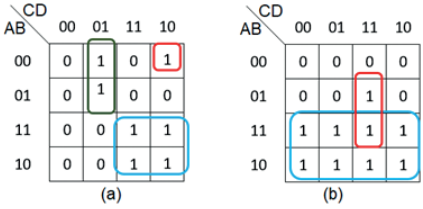


Figura 11. Formações de grupos. (a) Vemos três grupos, seguindo os tamanhos indicados, 1, 2 e 4 elementos; (b) formação utilizando o maior número de saídas, formado por 8 valores. Neste caso também está se seguindo a regra de grupos maiores, ou seja, em vez de criar um grupo com 1 elemento, foi intercalado com outro grupo, gerando um grupo de 2 elementos.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

- Quanto à posição dos mintermos, estes poderão ser formados por linhas, colunas ou "quadrados". É importante lembrar que somente devem ser formados grupos com mintermos adjacentes (formações em diagonal não são permitidas) (Figura 12).

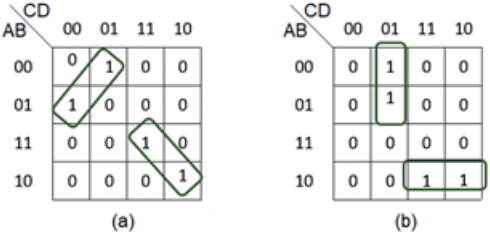


Figura 12. Exemplos de agrupamentos. (a) forma errada, unindo valores em diagonais. (b) forma correta, reunindo grupos adjacentes.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

A última parte do processo é a avaliação em si e a montagem da expressão. Considerando o mapa criado na Figura 13, podemos notar que ele possui três grupos, logo irá gerar 3 termos para a expressão. Cada grupo é avaliado em separado.

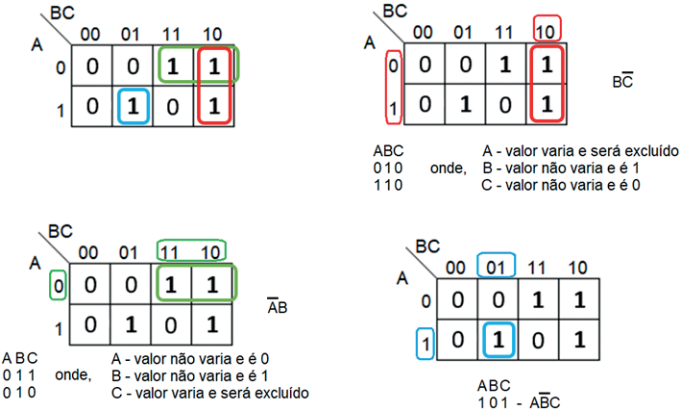


Figura 13. Mapa criado anteriormente e avaliação para cada um dos três grupos.
Fonte: Adaptada de Tocci, Widmer e Moss (2011).

Para grupos de um elemento, a expressão não irá simplificar termos, somente identificar o termo, para tanto observamos os valores dos bits da coluna — para cada valor 1, substituímos pela letra da variável correspondente àquela posição, e cada valor 0, substituímos pela respectiva variável negada.

Já para os casos em que encontramos dois ou mais valores, precisamos avaliar se ocorre variação nos bits; caso não ocorra, a variável é incluída na fórmula com seu respectivo valor negado ou não. Caso o valor varie, seja na coluna ou na linha, e uma variável apresente os valores 0 e 1, esta deve ser excluída do termo, ocorrendo a simplificação.

Ao final, basta reunir os termos identificados. Como estamos aplicando a soma de produtos, basta somar as expressões individuais para obter a expressão booleana correspondente, no caso $\bar{A}B + B\bar{C} + A\bar{B}C$.

Referências

ABE, J. M. *Introdução à lógica para a ciência da computação*. São Paulo: Arte & Ciência, 2002.

DAGHLIAN, J. *Lógica e álgebra de Boole*. 4. ed. São Paulo: Atlas, 1995.

DIAS, M. *Sistemas digitais*: princípios e prática. [*S. l.*]: FCA, 2013.

DORAN, R. W. The gray code. *Journal of Universal Computer Science*, [*s. l.*], v. 13, n. 11, p. 1573–1597, 2007. Disponível em: http://www.jucs.org/jucs_13_11/the_gray_code/jucs_13_11_1573_1597_doran.pdf. Acesso em: 4 mar. 2021.

TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas digitais*: princípios e aplicações. 11. ed. [*S. l.*]: Pearson, 2011.

Leitura recomendada

MACHADO, N. J.; CUNHA, M. O. *Lógica e linguagem cotidiana*: verdade, coerência, comunicação, argumentação. 3. ed. Belo Horizonte: Autêntica, 2007.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS